

BÁO CÁO THỰC HÀNH: KỸ NĂNG VIẾT PROMPT TRONG HỌC TẬP

Họ và tên: Mai Đăng Khánh

MSSV: 25020657

I. Lựa chọn và Phân tích Tác vụ Học tập (Tiêu chí 1)

Dưới đây là 3 tác vụ học tập điển hình được lựa chọn, kèm theo phân tích về bản chất và thách thức:

1. Tác vụ 1: Tóm tắt tài liệu học thuật (Chủ đề: Bài báo khoa học về ứng dụng AI trong Viễn thông)

(Nguồn: [1], [2], [3])

Artificial Intelligence Driven Channel Coding and Resource Optimization for Wireless Networks

Yasir Ali, Tayyab Manzoor, Huan Yang, Chenhang Yan, Yuanqing Xia

The ongoing evolution of 5G and its enhanced version, 5G+, has significantly transformed the telecommunications landscape, driving an unprecedented demand for ultra-high-speed data transmission, ultra-low latency, and resilient connectivity. These capabilities are essential for enabling mission-critical applications such as the Internet of Things, autonomous vehicles, and smart city infrastructures. This paper investigates the important role of Artificial Intelligence (AI) in addressing the key challenges faced by 5G/5G+ networks, including interference mitigation, dynamic resource allocation, and maintaining seamless network operation. The study particularly focuses on AI-driven innovations in coding theory, which offer advanced solutions to the limitations of conventional error correction and modulation techniques. By employing deep learning, reinforcement learning, and neural network-based approaches, this research demonstrates significant advancements in error correction performance, decoding efficiency, and adaptive transmission strategies. Additionally, the integration of AI with emerging technologies, such as massive multiple-input and multiple-output, intelligent reflecting surfaces, and privacy-enhancing mechanisms, is discussed, highlighting their potential to propel the next generation of wireless networks. This paper also provides insights into the transformative impact of AI on modern wireless communication, establishing a foundation for scalable, adaptive, and more efficient network architectures.

Comment: 5/5 Điểm

- Mục tiêu:** Nắm bắt nhanh các phương pháp và kết quả nghiên cứu mà không phải đọc toàn bộ tài liệu dài hàng chục trang.
- Thách thức:** AI thường dễ bị lan man, mất đi các thuật ngữ chuyên ngành quan trọng hoặc không phân tách rõ ràng giữa "vấn đề", "giải pháp" và "kết quả".

2. Tác vụ 2: Giải thích khái niệm phức tạp (Chủ đề: Con trỏ - Pointers và Cấp phát bộ nhớ động trong C)

- Mục tiêu:** Hiểu sâu về cách quản lý bộ nhớ (malloc/free) ở mức độ vật lý thay vì chỉ học vẹt cú pháp.
- Thách thức:** Đây là khái niệm rất trừu tượng. Nếu prompt không tốt, AI sẽ chỉ trả về định nghĩa sách giáo khoa khô khan, thiếu tính trực quan để người mới học lập trình có thể hình dung.

3. Tác vụ 3: Tạo bộ câu hỏi ôn tập (Chủ đề: Đạo hàm đa biến, Vector Gradient và Ma trận Hessian)

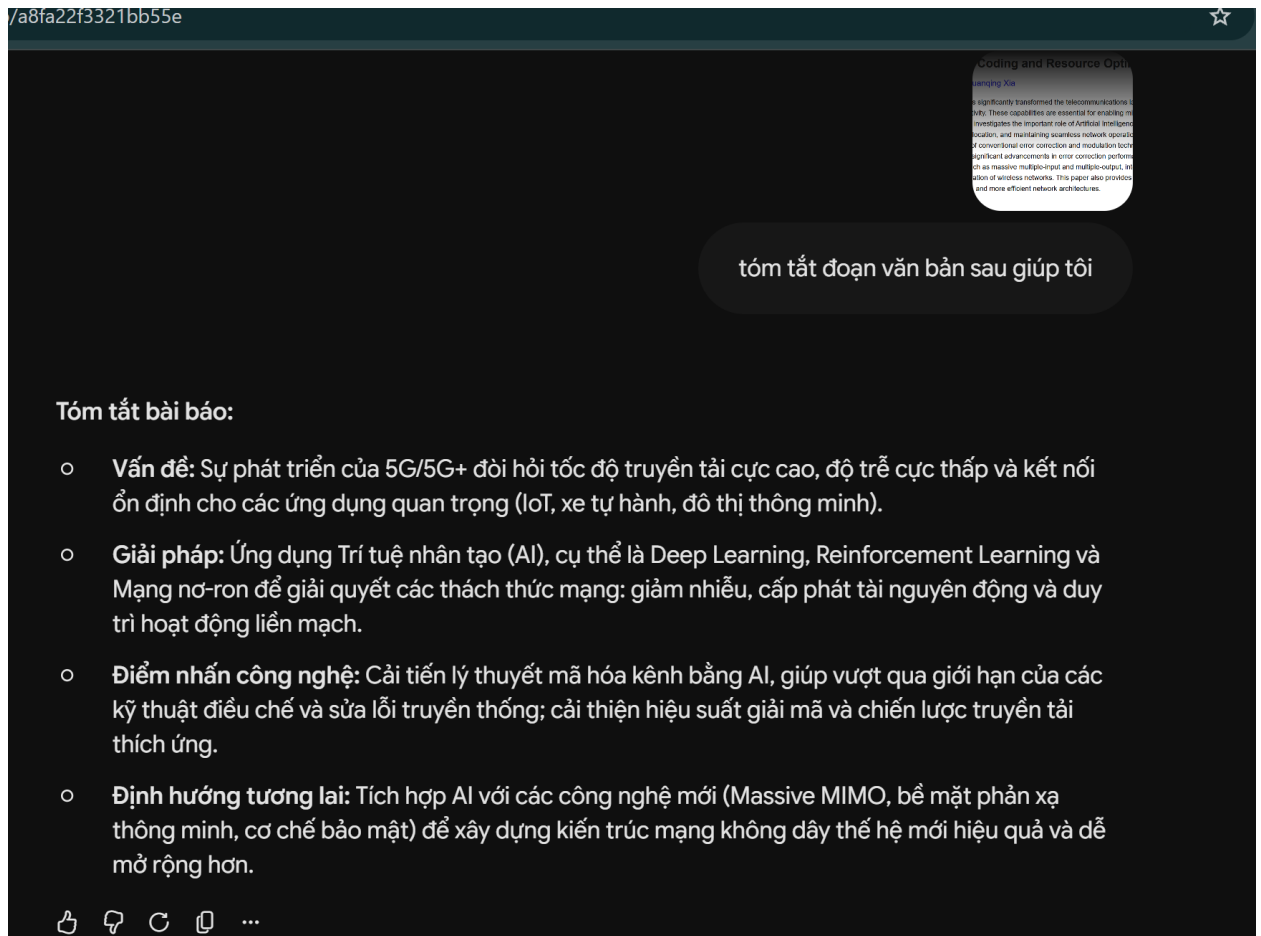
- Mục tiêu:** Tự kiểm tra năng lực tính toán và hiểu biết lý thuyết trước kỳ thi Toán kỹ thuật.

- *Thách thức:* AI có thể tạo ra các bài toán quá dễ hoặc giải sai các bước toán học phức tạp nếu không được định hướng phương pháp tư duy (Chain-of-thought) rõ ràng.

II. Xây dựng và Thử nghiệm các phiên bản Prompt (Tiêu chí 2 & 3)

Tác vụ 1: Tóm tắt bài đọc/tài liệu học thuật

- **Prompt Cơ bản:** "Tóm tắt đoạn văn bản sau giúp tôi:"



- **Prompt Cải tiến (Thêm cấu trúc và độ dài):** "Hãy tóm tắt bài báo khoa học dưới đây. Yêu cầu: Viết thành 3 đoạn văn ngắn gọn, nêu rõ :

- 1) Vấn đề nghiên cứu,
- 2) Phương pháp AI được đề xuất, và
- 3) Kết quả đạt được: "

đoạn văn ngắn gọn, nêu rõ :

- 1) Vấn đề nghiên cứu,
- 2) Phương pháp AI được đề xuất, và

Dưới đây là tóm tắt bài báo khoa học theo đúng 3 yêu cầu của bạn:

1) Vấn đề nghiên cứu:

Sự phát triển của mạng 5G và 5G+ đặt ra yêu cầu khắt khe về tốc độ dữ liệu siêu cao, độ trễ cực thấp và kết nối bền vững để phục vụ các ứng dụng trọng yếu (như IoT, xe tự hành, đô thị thông minh). Tuy nhiên, các kỹ thuật điều chế và sửa lỗi truyền thống đang bộc lộ nhiều hạn chế, gây khó khăn cho việc giảm thiểu nhiễu, cấp phát tài nguyên động và duy trì mạng hoạt động liên mạch.

2) Phương pháp AI được đề xuất:

Nghiên cứu ứng dụng Trí tuệ Nhân tạo (AI) để giải quyết các hạn chế của lý thuyết mã hóa kênh thông thường. Bài báo đề xuất sử dụng kết hợp các thuật toán học sâu (deep learning), học tăng cường (reinforcement learning) và các phương pháp dựa trên mạng nơ-ron (neural network-based approaches) để can thiệp vào quá trình tối ưu hóa tài nguyên mạng không dây.

3) Kết quả đạt được:

Việc triển khai AI đã mang lại những bước tiến đáng kể trong hiệu suất sửa lỗi, hiệu năng giải mã và các chiến lược truyền tải thích ứng. Hơn nữa, nghiên cứu chỉ ra rằng việc tích hợp AI với các công nghệ tiên tiến khác (như massive MIMO, bề mặt phản xạ thông minh) sẽ tạo ra một nền tảng vững chắc cho các kiến trúc mạng thế hệ tiếp theo, đảm bảo tính mở rộng, linh hoạt và hiệu quả tối đa.

👍 🗨️ 🔄 📄 ...

- **Prompt Nâng cao (Role-prompting + Chain-of-thought + Format):** "Hãy đóng vai một nhà nghiên cứu cấp cao chuyên ngành Điện tử Viễn thông. Nhiệm vụ của bạn là tóm tắt bài báo học thuật dưới đây cho một sinh viên đại học năm 2 để họ dễ hiểu nhất.

Hãy thực hiện suy luận theo các bước sau:

Bước 1: Trích xuất các từ khóa chuyên ngành cốt lõi.

Bước 2: Tóm tắt mục tiêu nghiên cứu.

Bước 3: Đánh giá ngắn gọn tính ứng dụng thực tiễn của phương pháp. Yêu cầu định dạng: Trình bày dưới dạng Bullet points (gạch đầu dòng), in đậm các thuật ngữ quan trọng. Bài báo:"

- Bước 1: Các từ khóa chuyên ngành cốt lõi
 - 5G/5G+
 - Trí tuệ Nhân tạo (AI)
 - Mã hóa kênh (Channel Coding)
 - Cấp phát tài nguyên động (Dynamic Resource Allocation)
 - Học sâu (Deep Learning) và Học tăng cường (Reinforcement Learning)
 - Massive MIMO
- Bước 2: Tóm tắt mục tiêu nghiên cứu
 - Nghiên cứu ứng dụng AI để giải quyết các thách thức vận hành của mạng 5G/5G+ như giảm nhiễu và cấp phát tài nguyên động.
 - Tập trung dùng AI để cải tiến lý thuyết mã hóa, khắc phục những hạn chế của các kỹ thuật sửa lỗi và điều chế truyền thống.
- Bước 3: Đánh giá tính ứng dụng thực tiễn
 - Tính ứng dụng trực tiếp cao nhờ cải thiện rõ rệt hiệu suất sửa lỗi, hiệu năng giải mã và tối ưu hóa chiến lược truyền tải thích ứng.
 - Tạo nền tảng kiến trúc mạng để mở rộng khi kết hợp AI với các công nghệ phần cứng tiên tiến hiện nay như Massive MIMO và bề mặt phản xạ thông minh.

👍 💬 🔄 📄 ...

Tác vụ 2: Giải thích một khái niệm phức tạp

- **Prompt Cơ bản:** "Giải thích về con trỏ (pointer) và hàm malloc/free trong ngôn ngữ C."

Con trỏ (Pointer)

- Là gì: Biến lưu địa chỉ ô nhớ của biến khác.
- Toán tử `&` : Lấy địa chỉ của biến.
- Toán tử `*` : Lấy giá trị tại địa chỉ mà con trỏ trỏ tới.

Malloc & Free (Quản lý bộ nhớ Heap)

- `malloc(size)` : Cấp phát vùng nhớ động (tính bằng byte). Trả về con trỏ trỏ đến vùng nhớ đó (hoặc `NULL` nếu hết bộ nhớ).
- `free(ptr)` : Giải phóng vùng nhớ đã cấp phát để tránh rò rỉ bộ nhớ (Memory Leak).

- **Prompt Cải tiến (Yêu cầu thêm ví dụ):** > "Hãy giải thích chi tiết về khái niệm con trỏ (pointer) và cách sử dụng cấp phát bộ nhớ động (malloc, free) trong C. Vui lòng cung cấp một ví dụ code minh họa."

Khái niệm Con trỏ (Pointer)

Con trỏ là một biến đặc biệt dùng để lưu trữ địa chỉ bộ nhớ của một biến khác, thay vì lưu trữ giá trị trực tiếp.

- Toán tử `&` (Address-of): Lấy địa chỉ của một biến trong bộ nhớ.
- Toán tử `*` (Dereference): Lấy giá trị đang lưu tại địa chỉ mà con trỏ trỏ tới.

Cấp phát bộ nhớ động (Dynamic Memory Allocation)

Các biến thông thường được cấp phát tĩnh trên vùng nhớ Stack (kích thước cố định từ lúc biên dịch, tự động giải phóng). Cấp phát động cho phép xin cấp phát bộ nhớ trên vùng Heap trong quá trình chương trình đang chạy (runtime), giúp thay đổi kích thước dữ liệu linh hoạt.

1. Hàm `malloc` (Memory Allocation)

- **Mục đích:** Xin cấp phát một khối bộ nhớ có kích thước cụ thể (tính bằng byte).
- **Cú pháp:** `void* malloc(size_t size);`
- **Cơ chế hoạt động:** * Trả về một con trỏ trỏ tới byte đầu tiên của khối nhớ vừa cấp phát.
 - Do `malloc` trả về `void*`, cần ép kiểu (typecast) về đúng loại con trỏ mong muốn (ví dụ `int*`).
 - Nếu hệ thống hết bộ nhớ, trả về `NULL`.
 - Vùng nhớ cấp phát chứa các giá trị rác ban đầu.

2. Hàm `free`

- **Mục đích:** Giải phóng và trả lại vùng nhớ động cho hệ thống khi không còn sử dụng.
- **Cú pháp:** `void free(void* ptr);`
- **Tầm quan trọng:** Bắt buộc phải gọi. Nếu không, chương trình sẽ chiếm dụng bộ nhớ vĩnh viễn cho đến khi tắt, gây rò rỉ bộ nhớ (Memory Leak).
- **Lưu ý:** Sau khi `free`, con trỏ vẫn giữ địa chỉ cũ nhưng vùng nhớ đó không còn hợp lệ. Cần gán con trỏ bằng `NULL` để tránh lỗi con trỏ lơ lửng (Dangling Pointer).

Ví dụ Code minh họa

Chương trình dưới đây sử dụng con trỏ và `malloc` để tạo một mảng số nguyên có kích thước được quyết định lúc runtime, thao tác trên nó, và giải phóng nó an toàn.

Ví dụ Code minh họa

Chương trình dưới đây sử dụng con trỏ và `malloc` để tạo một mảng số nguyên có kích thước được quyết định lúc runtime, thao tác trên nó, và giải phóng nó an toàn.

```
C

#include <stdio.h>
#include <stdlib.h> // Thư viện chứa hàm malloc và free

int main() {
    int n = 5;
    int *arr; // Khai báo con trỏ

    // 1. Cấp phát vùng nhớ cho n phần tử kiểu int
    // sizeof(int) thường là 4 bytes. Tổng cần xin: 5 * 4 = 20 bytes.
    arr = (int*) malloc(n * sizeof(int));

    // Kiểm tra cấp phát có thành công không
    if (arr == NULL) {
        printf("Lỗi: Không thể cấp phát bộ nhớ.\n");
        return 1;
    }

    // 2. Thao tác trên vùng nhớ động thông qua con trỏ
    printf("Dữ liệu mảng: ");
    for (int i = 0; i < n; i++) {
        // *(arr + i) tương đương với arr[i]
        *(arr + i) = (i + 1) * 10;
        printf("%d ", *(arr + i));
    }
    printf("\n");

    // 3. Giải phóng bộ nhớ
    free(arr);

    // 4. Reset con trỏ để đảm bảo an toàn
    arr = NULL;

    return 0;
}
```

- **Prompt Nâng cao (Role-prompting + Analogy + Strict Output Constraints):** >
"Đóng vai một kỹ sư phần mềm chuyên lập trình hệ thống Linux. Hãy giải thích khái niệm 'con trỏ' và 'cấp phát bộ nhớ động' (malloc/free) trong C cho một người mới bắt đầu."

Chào bạn. Dưới góc độ lập trình hệ thống Linux, quản lý bộ nhớ là vấn đề sống còn. Đây là bản chất của hai khái niệm này một cách chính xác và thực tế nhất:

1. Con trỏ (Pointer)

Tương tự như RAM máy tính là một mảng khổng lồ các ô nhớ, mỗi ô có một địa chỉ vật lý (ví dụ: `0x7fff...`).

- **Định nghĩa:** Con trỏ là một biến đặc biệt chỉ dùng để lưu trữ địa chỉ của ô nhớ khác, không lưu dữ liệu thực tế.
- **Thực tế hệ thống:** Thay vì phải copy một cục dữ liệu lớn (ví dụ struct 1GB) truyền qua lại giữa các hàm, bạn chỉ cần đưa "địa chỉ" của nó (8 bytes trên hệ thống 64-bit). Cực kỳ tối ưu.

2. Cấp phát bộ nhớ động (malloc / free)

Trong Linux, bộ nhớ của một tiến trình (process) chia làm nhiều vùng. Hai vùng quan trọng nhất chứa dữ liệu là **Stack** và **Heap**. Biến thông thường nằm trên Stack (OS tự cấp phát, tự thu hồi, nhưng dung lượng rất giới hạn, thường 8MB).

Nếu bạn cần lưu trữ dữ liệu lớn hoặc chưa biết kích thước trước khi chạy, bạn phải dùng vùng **Heap**:

- `malloc(size)` : Gửi yêu cầu (qua system call như `brk` hoặc `mmap`) xin Linux Kernel cấp cho một khối bộ nhớ trên Heap. Kernel tìm chỗ trống, đánh dấu là đã dùng, và trả về con trỏ trỏ tới địa chỉ đầu tiên của khối đó.
- `free(ptr)` : Lệnh báo cho Kernel biết "tôi đã dùng xong, hãy thu hồi khối nhớ này".
- **Cảnh báo:** Nếu bạn dùng `malloc` mà quên `free`, tiến trình sẽ bị **Memory Leak** (rò rỉ bộ nhớ). Chạy càng lâu càng ngốn RAM, đến khi cạn kiệt, tính năng **OOM-Killer** (Out-Of-Memory) của Linux sẽ tự động "bắn hạ" (kill) ứng dụng của bạn để bảo vệ hệ thống.

Code minh họa chuẩn mực an toàn

```
c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // 1. Khai báo con trỏ
    int *p = NULL;

    // 2. Xin Kernel cấp phát 4 bytes (kích thước 1 số int) trên Heap
    p = (int*) malloc(sizeof(int));
    if (p == NULL) {
        printf("Kernel từ chối cấp phát bộ nhớ (Hết RAM)!\n");
        return 1;
    }

    // 3. Sử dụng vùng nhớ thông qua con trỏ
    *p = 42;
    printf("Giá trị lưu tại Heap: %d\n", *p);

    // 4. Hoàn trả vùng nhớ cho hệ thống
    free(p);

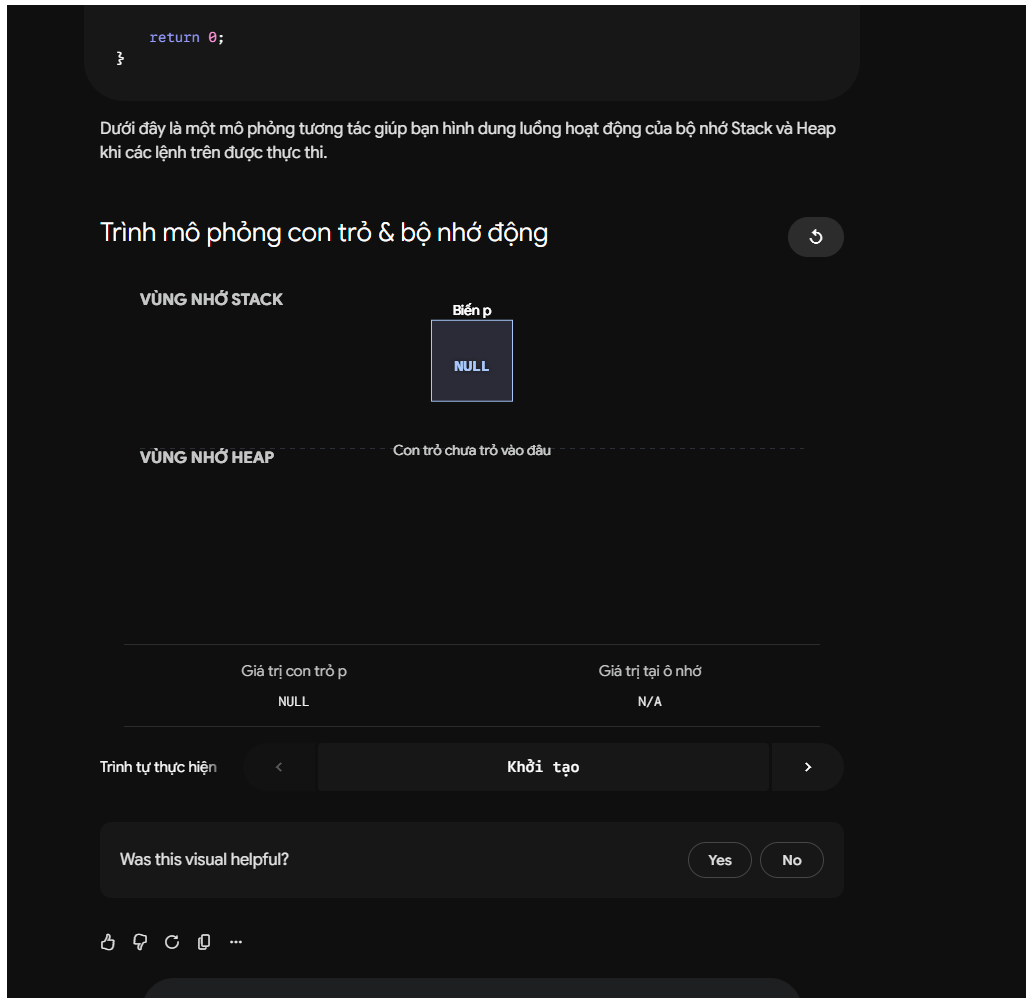
    // 5. Triệt tiêu con trỏ lơ lửng (Dangling pointer)
    p = NULL;

    return 0;
}
```

+ Ask Gemini

Pro





Tác vụ 3: Tạo bộ câu hỏi ôn tập

- **Prompt Cơ bản:** > "Tạo cho tôi bài tập về Vector Gradient và Ma trận Hessian."

Dưới đây là 2 bài tập tính Vector Gradient và Ma trận Hessian kèm đáp án:

Bài tập 1: Hàm 2 biến

Đề bài: Cho hàm số $f(x, y) = x^3 - 3xy^2 + y^3$. Hãy tìm Vector Gradient ∇f và Ma trận Hessian H .

Đáp án:

1. Vector Gradient (∇f)

Tính các đạo hàm riêng bậc 1:

- $\frac{\partial f}{\partial x} = 3x^2 - 3y^2$
- $\frac{\partial f}{\partial y} = -6xy + 3y^2$

$$\nabla f = \begin{bmatrix} 3x^2 - 3y^2 \\ -6xy + 3y^2 \end{bmatrix}$$

2. Ma trận Hessian (H)

Tính các đạo hàm riêng bậc 2:

- $f_{xx} = \frac{\partial^2 f}{\partial x^2} = 6x$
- $f_{xy} = \frac{\partial^2 f}{\partial x \partial y} = -6y$
- $f_{yx} = \frac{\partial^2 f}{\partial y \partial x} = -6y$
- $f_{yy} = \frac{\partial^2 f}{\partial y^2} = -6x + 6y$

$$H = \begin{bmatrix} 6x & -6y \\ -6y & -6x + 6y \end{bmatrix}$$

Bài tập 2: Hàm 3 biến (Tính tại một điểm)

Đề bài: Cho hàm số $f(x, y, z) = x^2y + y^2z$. Hãy tính ∇f và H tại điểm $P(1, 1, 1)$.

Đáp án:

1. Vector Gradient tại $P(1, 1, 1)$

- $f_x = 2xy$
- $f_y = x^2 + 2yz$
- $f_z = y^2$

$$\nabla f(x, y, z) = \begin{bmatrix} 2xy \\ x^2 + 2yz \\ y^2 \end{bmatrix}$$

- Prompt Cải tiến (Định lượng và yêu cầu đáp án):** > "Hãy tạo một bộ gồm 3 bài tập tự luận về Đạo hàm đa biến, bao gồm tính toán Vector Gradient và Ma trận Hessian. Cung cấp đáp án chi tiết ở cuối."

Dưới đây là bộ 3 bài tập tự luận về Đạo hàm đa biến (Vector Gradient và Ma trận Hessian) được sắp xếp từ cơ bản đến nâng cao, kèm theo hướng dẫn giải chi tiết.

Phần Đề Bài

Bài 1: Hàm đa thức 2 biến cơ bản

Cho hàm số $f(x, y) = 2x^3 - 3x^2y + y^4$.

- 1. Tìm Vector Gradient ∇f và Ma trận Hessian H tổng quát theo x và y .
- 2. Tính ∇f và H tại điểm $A(1, -1)$.

Bài 2: Hàm 3 biến đối xứng

Cho hàm số $g(x, y, z) = x^2y + y^2z + z^2x$.

Hãy xác định Vector Gradient $\nabla g(x, y, z)$ và Ma trận Hessian $H(x, y, z)$.

Bài 3: Ứng dụng tìm điểm tới hạn

Cho hàm số $h(x, y) = x^3 + y^3 - 3xy$.

- 1. Tìm Vector Gradient ∇h .
- 2. Giải phương trình $\nabla h = 0$ để tìm các điểm tới hạn của hàm số.
- 3. Lập Ma trận Hessian H và tính giá trị của H tại các điểm tới hạn vừa tìm được.

Phần Đáp Án Chi Tiết

Đáp án Bài 1

1. Tính toán tổng quát:

Tính các đạo hàm riêng bậc 1:

- $\frac{\partial f}{\partial x} = 6x^2 - 6xy$
- $\frac{\partial f}{\partial y} = -3x^2 + 4y^3$

Vector Gradient:

$$\nabla f(x, y) = \begin{bmatrix} 6x^2 - 6xy \\ -3x^2 + 4y^3 \end{bmatrix}$$

Tính các đạo hàm riêng bậc 2:

- $f_{xx} = \frac{\partial}{\partial x}(6x^2 - 6xy) = 12x - 6y$
- $f_{xy} = f_{yx} = \frac{\partial}{\partial y}(6x^2 - 6xy) = -6x$
- $f_{yy} = \frac{\partial}{\partial y}(-3x^2 + 4y^3) = 12y^2$

Ma trận Hessian:

$$H(x, y) = \begin{bmatrix} 12x - 6y & -6x \\ -6x & 12y^2 \end{bmatrix}$$

+ Ask Gemini

Pro



- $f_{yy} = \frac{\partial}{\partial y}(-3x^2 + 4y^3) = 12y^2$

Ma trận Hessian:

$$H(x, y) = \begin{bmatrix} 12x - 6y & -6x \\ -6x & 12y^2 \end{bmatrix}$$

2. Tính tại điểm $A(1, -1)$:

Thay $x = 1, y = -1$ vào các kết quả trên:

$$\nabla f(1, -1) = \begin{bmatrix} 6(1)^2 - 6(1)(-1) \\ -3(1)^2 + 4(-1)^3 \end{bmatrix} = \begin{bmatrix} 12 \\ -7 \end{bmatrix}$$

$$H(1, -1) = \begin{bmatrix} 12(1) - 6(-1) & -6(1) \\ -6(1) & 12(-1)^2 \end{bmatrix} = \begin{bmatrix} 18 & -6 \\ -6 & 12 \end{bmatrix}$$

Đáp án Bài 2

1. Vector Gradient:

Tính đạo hàm riêng bậc 1 theo từng biến:

- $g_x = 2xy + z^2$
- $g_y = x^2 + 2yz$
- $g_z = y^2 + 2zx$

$$\nabla g(x, y, z) = \begin{bmatrix} 2xy + z^2 \\ x^2 + 2yz \\ y^2 + 2zx \end{bmatrix}$$

2. Ma trận Hessian:

Tính các đạo hàm riêng bậc 2:

- $g_{xx} = 2y, g_{xy} = 2x, g_{xz} = 2z$
- $g_{yx} = 2x, g_{yy} = 2z, g_{yz} = 2y$
- $g_{zx} = 2z, g_{zy} = 2y, g_{zz} = 2x$

Lưu ý tính đối xứng ($g_{xy} = g_{yx}$, v.v.), ta có Ma trận Hessian 3x3:

$$H(x, y, z) = \begin{bmatrix} 2y & 2x & 2z \\ 2x & 2z & 2y \\ 2z & 2y & 2x \end{bmatrix}$$

Đáp án Bài 3

1. Vector Gradient ∇h :

- $h_x = 3x^2 - 3y$
- $h_y = 3y^2 - 3x$

$$\nabla h(x, y) = \begin{bmatrix} 3x^2 - 3y \\ 3y^2 - 3x \end{bmatrix}$$

2. Tìm điểm tới hạn:

Cho $\nabla h = 0$, ta có hệ phương trình:

$$1 \quad 3x^2 - 3y = 0 \Rightarrow y = x^2$$

$$2 \quad 3y^2 - 3x = 0 \Rightarrow x = y^2$$

Thay (1) vào (2): $x = (x^2)^2 \Rightarrow x^4 - x = 0 \Rightarrow x(x^3 - 1) = 0$

Từ đó ta có nghiệm $x = 0$ hoặc $x = 1$.

- Với $x = 0 \Rightarrow y = 0$. Điểm $M_1(0, 0)$.
- Với $x = 1 \Rightarrow y = 1$. Điểm $M_2(1, 1)$.

Hàm số có 2 điểm tới hạn là $(0, 0)$ và $(1, 1)$.

3. Ma trận Hessian tại các điểm tới hạn:

Đạo hàm bậc 2:

- $h_{xx} = 6x$
- $h_{xy} = h_{yx} = -3$
- $h_{yy} = 6y$

Ma trận Hessian tổng quát:

$$H(x, y) = \begin{bmatrix} 6x & -3 \\ -3 & 6y \end{bmatrix}$$

Tại điểm $M_1(0, 0)$:

$$H(0, 0) = \begin{bmatrix} 0 & -3 \\ -3 & 0 \end{bmatrix}$$

Tại điểm $M_2(1, 1)$:

$$H(1, 1) = \begin{bmatrix} 6 & -3 \\ -3 & 6 \end{bmatrix}$$

👉 🔍 ⌂ ☰ ...

- **Prompt Nâng cao (Role-prompting + Phân cấp độ khó + Few-shot/Cấu trúc đáp án):** > "Đóng vai là giảng viên môn Toán Kỹ thuật cao cấp. Hãy tạo một bài kiểm tra ôn tập ngắn về chủ đề 'Đạo hàm nhiều biến và Vector Gradient'.

Yêu cầu cấu trúc:

- Câu 1 (Dễ): Tính đạo hàm riêng cấp 1 của một hàm số 2 biến.

- Câu 2 (Trung bình): Tìm Vector Gradient của một hàm 3 biến tại một điểm cụ thể.
- Câu 3 (Khó): Tính Ma trận Hessian và xác định cực trị. Yêu cầu định dạng: Trình bày toàn bộ các công thức toán học bằng định dạng LaTeX (sử dụng dấu \$ cho công thức trong dòng và \$\$ cho công thức độc lập). Yêu cầu lời giải: Phần lời giải phải được trình bày theo từng bước (step-by-step reasoning), giải thích lý do áp dụng công thức ở mỗi bước."

Đề Kiểm Tra Ôn Tập: Đạo Hàm Nhiều Biến & Vector Gradient

Câu 1 (Dễ)

Tính các đạo hàm riêng cấp 1 của hàm số $f(x, y) = x^3y^2 - 2x + e^y$.

Câu 2 (Trung bình)

Tìm Vector Gradient của hàm số $g(x, y, z) = x^2y + yz^2$ tại điểm $P(1, 2, -1)$.

Câu 3 (Khó)

Cho hàm số $h(x, y) = x^2 + xy + y^2 - 3x$. Lập Ma trận Hessian và xác định các điểm cực trị (nếu có).

Lời Giải Chi Tiết

Câu 1

Bước 1: Tính đạo hàm theo x

Lý do: Để tính đạo hàm riêng theo x , ta coi y là hằng số và áp dụng quy tắc đạo hàm cơ bản của biến x .

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x}(x^3y^2) - \frac{\partial}{\partial x}(2x) + \frac{\partial}{\partial x}(e^y)$$

$$\frac{\partial f}{\partial x} = 3x^2y^2 - 2 + 0 = 3x^2y^2 - 2$$

Bước 2: Tính đạo hàm theo y

Lý do: Tương tự, coi x là hằng số và đạo hàm theo biến y .

$$\frac{\partial f}{\partial y} = \frac{\partial}{\partial y}(x^3y^2) - \frac{\partial}{\partial y}(2x) + \frac{\partial}{\partial y}(e^y)$$

$$\frac{\partial f}{\partial y} = 2x^3y - 0 + e^y = 2x^3y + e^y$$

Câu 2

Bước 1: Thiết lập công thức Vector Gradient

Lý do: Vector Gradient của hàm 3 biến là vector chứa các đạo hàm riêng cấp 1 của từng biến.

$$\nabla g(x, y, z) = \begin{bmatrix} g_x \\ g_y \\ g_z \end{bmatrix}$$

Bước 2: Tính các đạo hàm riêng tổng quát

+ Ask Gemini

Pro ~



$$\nabla g(x, y, z) = \begin{bmatrix} g_x \\ g_y \\ g_z \end{bmatrix}$$

Bước 2: Tính các đạo hàm riêng tổng quát

Lý do: Cần có phương trình tổng quát trước khi thay số để tránh sai sót.

- $g_x = \frac{\partial}{\partial x}(x^2y + yz^2) = 2xy$
- $g_y = \frac{\partial}{\partial y}(x^2y + yz^2) = x^2 + z^2$
- $g_z = \frac{\partial}{\partial z}(x^2y + yz^2) = 2yz$

Bước 3: Thay tọa độ điểm $P(1, 2, -1)$

Lý do: Để yêu cầu tính Gradient tại một điểm cụ thể, ta thế $x = 1, y = 2, z = -1$ vào các kết quả trên.

- $g_x(1, 2, -1) = 2(1)(2) = 4$
- $g_y(1, 2, -1) = 1^2 + (-1)^2 = 2$
- $g_z(1, 2, -1) = 2(2)(-1) = -4$

$$\nabla g(1, 2, -1) = \begin{bmatrix} 4 \\ 2 \\ -4 \end{bmatrix}$$

Câu 3

Bước 1: Tìm điểm tới hạn

Lý do: Theo điều kiện cần của cực trị, điểm cực trị phải nằm trong tập các điểm tới hạn (nơi $\nabla h = 0$).

Tính Gradient và cho bằng 0:

- 1 $h_x = 2x + y - 3 = 0$
- 2 $h_y = x + 2y = 0 \Rightarrow x = -2y$

Thế $x = -2y$ vào phương trình 1:

$$2(-2y) + y - 3 = 0 \Rightarrow -3y = 3 \Rightarrow y = -1$$

Suy ra $x = 2$.

Ta có một điểm tới hạn là $M(2, -1)$.

Bước 2: Lập Ma trận Hessian H

Lý do: Áp dụng điều kiện đủ (tiêu chuẩn đạo hàm bậc 2) để phân loại điểm tới hạn thành cực đại, cực tiểu hoặc điểm yên ngựa.

Tính các đạo hàm riêng cấp 2:

- $h_{xx} = \frac{\partial}{\partial x}(2x + y - 3) = 2$
- $h_{xy} = h_{yx} = \frac{\partial}{\partial y}(2x + y - 3) = 1$
- $h_{yy} = \frac{\partial}{\partial y}(x + 2y) = 2$

Lý do: Các cơ phương trình tổng quát được khi thay số để tránh sai sót.

- $g_x = \frac{\partial}{\partial x}(x^2y + yz^2) = 2xy$
- $g_y = \frac{\partial}{\partial y}(x^2y + yz^2) = x^2 + z^2$
- $g_z = \frac{\partial}{\partial z}(x^2y + yz^2) = 2yz$

Bước 3: Thay tọa độ điểm $P(1, 2, -1)$

Lý do: Để yêu cầu tính Gradient tại một điểm cụ thể, ta thế $x = 1, y = 2, z = -1$ vào các kết quả trên.

- $g_x(1, 2, -1) = 2(1)(2) = 4$
- $g_y(1, 2, -1) = 1^2 + (-1)^2 = 2$
- $g_z(1, 2, -1) = 2(2)(-1) = -4$

$$\nabla g(1, 2, -1) = \begin{bmatrix} 4 \\ 2 \\ -4 \end{bmatrix}$$

Câu 3

Bước 1: Tìm điểm tới hạn

Lý do: Theo điều kiện cần của cực trị, điểm cực trị phải nằm trong tập các điểm tới hạn (nơi $\nabla h = 0$).

Tính Gradient và cho bằng 0:

1 $h_x = 2x + y - 3 = 0$

2 $h_y = x + 2y = 0 \Rightarrow x = -2y$

Thế $x = -2y$ vào phương trình 1:

$$2(-2y) + y - 3 = 0 \Rightarrow -3y = 3 \Rightarrow y = -1$$

Suy ra $x = 2$.

Ta có một điểm tới hạn là $M(2, -1)$.

Bước 2: Lập Ma trận Hessian H

Lý do: Áp dụng điều kiện đủ (tiêu chuẩn đạo hàm bậc 2) để phân loại điểm tới hạn thành cực đại, cực tiểu hoặc điểm yên ngựa.

Tính các đạo hàm riêng cấp 2:

- $h_{xx} = \frac{\partial}{\partial x}(2x + y - 3) = 2$
- $h_{xy} = h_{yx} = \frac{\partial}{\partial y}(2x + y - 3) = 1$
- $h_{yy} = \frac{\partial}{\partial y}(x + 2y) = 2$

$$H = \begin{bmatrix} h_{xx} & h_{xy} \\ h_{yx} & h_{yy} \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Bước 3: Xác định cực trị tại $M(2, -1)$

Lý do: Dựa vào định thức Δ (hoặc D) của ma trận Hessian và dấu của h_{xx} để kết luận tính chất điểm.

+ Ask Gemini

Pro v



• III. Bảng So sánh và Phân tích Hiệu quả (Tiêu chí 4)

Tác vụ	Kết quả Prompt Cơ bản	Kết quả Prompt Nâng cao	Phân tích lý do hiệu quả (Tại sao bản Nâng cao tốt hơn?)
Tóm tắt	Trả về một khối văn bản đặc chữ, sao chép y nguyên các câu dài trong bản gốc. Khó đọc.	Trình bày rõ ràng theo từng gạch đầu dòng. Có insight đánh giá tính ứng dụng.	Prompt nâng cao sử dụng Role-prompting thiết lập văn phong chuyên nghiệp nhưng dễ hiểu. Kỹ thuật Chain-of-thought ép mô hình phải qua bước trích xuất từ khóa trước khi tóm tắt, giúp giữ được độ chính xác của thuật ngữ Viễn thông.
Giải thích	Đưa ra định nghĩa lý thuyết khô khan (ví dụ: "con trỏ là biến lưu địa chỉ bộ nhớ"). Code bị kèm theo giải thích dài dòng.	Khái niệm trở nên trực quan nhờ phép ẩn dụ (địa chỉ nhà). Code được xuất ra gọn gàng, đúng yêu cầu khắt khe.	Áp dụng kỹ thuật Analogy (Ẩn dụ) giúp não bộ người học liên kết kiến thức trừu tượng với vật lý. Việc đưa ra Ràng buộc đầu ra (Strict Constraints) "chỉ xuất code, không phụ chú" giúp tối ưu hóa thời gian đọc cho người chỉ cần tham khảo mã nguồn.
Tạo bài tập	Đưa ra bài tập ngẫu nhiên, không có phân loại độ khó, công thức toán học hiển thị lỗi font hoặc khó nhìn.	Phân bổ độ khó khoa học (Dễ - Trung bình - Khó). Công thức Toán học hiển thị cực kỳ chuyên nghiệp và chuẩn mực. Lời giải đi theo từng bước.	Sử dụng yêu cầu định dạng chuyên biệt (LaTeX cho Toán học) giúp tài liệu đạt chuẩn học thuật. Kỹ thuật định hướng lời giải Step-by-step reasoning ép AI giảm thiểu ảo giác (hallucination) và tránh sai sót trong tính toán các ma trận phức tạp.

IV. Tổng hợp Nguyên tắc và Mẹo viết Prompt (Tiêu chí 5)

Từ những thực nghiệm trên, ta có thể rút ra "Bộ 4 nguyên tắc vàng" (Mô hình C.A.R.E) để viết prompt phục vụ học tập khối ngành Kỹ thuật:

1. **C - Context (Bối cảnh & Đóng vai):** Luôn cung cấp vai trò cho AI (ví dụ: "Giảng viên Toán", "Kỹ sư C hệ thống"). Điều này giúp AI tự động điều chỉnh ngôn từ, mức độ học thuật và cách tiếp cận vấn đề.

2. **A - Action (Hành động & Phân tách bước):** Đừng giao một việc lớn. Hãy dùng kỹ thuật *Chain-of-thought* (Suy nghĩ theo chuỗi) để chia nhỏ yêu cầu: "Đầu tiên hãy làm X, sau đó làm Y, cuối cùng làm Z". Điều này đặc biệt quan trọng với các môn cần tính toán logic.
3. **R - Rules & Constraints (Ràng buộc):** Đưa ra các quy tắc chặt chẽ để định hình đầu ra. Ví dụ: "Chỉ in ra mã nguồn, không viết bình luận", hoặc "Bắt buộc sử dụng phép ẩn dụ".
4. **E - Example & Format (Định dạng):** Quy định rõ hình thức trình bày (Bullet points, Bảng, sử dụng LaTeX cho công thức toán) để kết quả trả về có thể sử dụng trực tiếp vào tài liệu ôn tập cá nhân mà không cần chỉnh sửa lại.